

This is the definitive Application Architecture Blueprint for the V1 DTE Platform. It dictates the structural boundaries, dependency rules, and interaction styles required to strictly implement the provided domain, data, and security specifications.

This blueprint focuses entirely on software architecture, intentionally omitting infrastructure, deployment, and framework details to ensure multi-year sustainability and alignment with the Ruthless Scope Cut.

SECTION 1 — Architectural Principles

The platform architecture is governed by immutable principles that guarantee regulatory compliance, defensibility, and multi-year sustainability:

- **Strict Separation of Software from Regulation:** Regulatory rules, MH catalogs, and schemas are strictly treated as data updates, never hardcoded into application execution paths. This ensures the platform can absorb Ministry of Finance updates without requiring recompilation or code deployment.
- **The Snapshot Law:** A validated fiscal document is a photograph of facts. Master data (issuer, receptor, items) is copied entirely by value into immutable JSONB structures. Retroactive corruption via live master data updates is physically impossible.
- **Append-Only Historical Truth:** History is immutable. Evidence artifacts and audit entries are append-only. The application never attempts to overwrite or delete regulatory artifacts, even for rejected or invalidated documents.
- **Cryptographic Verifiability & Defensibility:** The platform assumes internal actors or processes may attempt to alter history. To counter this, it utilizes tamper-evident hash chaining for audit trails and cryptographically hashed evidence bundles.
- **Least Privilege Execution:** The application assumes minimal necessary rights. The database roles used for operational execution explicitly lack UPDATE and DELETE privileges on evidence and audit tables.

SECTION 2 — Architectural Style

The application embraces a strict blend of **Domain-Driven Design (DDD)** and **Command/Query Responsibility Segregation (CQRS)** to isolate complex state mutations from read operations.

- **Overall Application Style:** The system is partitioned into discrete Bounded Contexts holding distinct Aggregate Roots. All state mutations are handled as explicit Commands, strictly separated from Data-Retrieval Queries (CQRS).
- **Module Interaction Style:** * *Business operations* (composition, validation, signing) are executed synchronously, returning immediate success or failure.
 - *Integration operations* (MH transmission) are executed asynchronously to decouple the user experience from API instability.
- **Transaction Boundaries:** Transactions are strictly bound to the Aggregate Root. Cross-aggregate consistency is achieved through asynchronous side-effects and explicit domain events, never through distributed or sprawling database transactions.

SECTION 3 — Module Map

The architecture is divided into the following isolated modules, each strictly adhering to the V1 domain boundaries.

1. Issuance (Core Engine)

- **Responsibilities:** Manages the core document state machine, assigns legal identity, interprets MH outcomes, and orchestrates signing.
- **Ownership:** FiscalDocument, ControlNumberSeries, TransmissionRecord.
- **Forbidden:** Managing master data, cryptographic key custody, or invalidation logic.

2. Invalidation

- **Responsibilities:** Lawfully voids an already-accepted document via plain invalidation without replacement.
- **Ownership:** InvalidationEvent.
- **Forbidden:** Altering the original document's signed payload or creating replacement documents.

3. Contingency

- **Responsibilities:** Governs offline issuance periods, holds documents, and orchestrates legal window reconciliation once connectivity returns.
- **Ownership:** ContingencyEpisode.
- **Forbidden:** Deciding document content or altering signing logic.

4. Parties & Counterparties (Catalog)

- **Responsibilities:** Maintains reusable receptor master data and validates tax identity formats.
- **Ownership:** Counterparty.
- **Forbidden:** Managing Receptor Snapshots inside historical documents.

5. Product & Catalog

- **Responsibilities:** Maintains item master data and default tax behaviors.
- **Ownership:** Item.
- **Forbidden:** Owning Document Lines or historical pricing.

6. Evidence & Archive

- **Responsibilities:** Preserves the append-only vault of cryptographic proofs (payloads, stamps, PDFs).
- **Ownership:** EvidenceBundle.
- **Forbidden:** Mutating business state or altering artifacts.

7. Audit Trail

- **Responsibilities:** Appends tamper-evident, hash-chained records of all significant system actions.
- **Ownership:** AuditTrail.
- **Forbidden:** Executing business decisions or mutating other contexts.

8. Access & Identity

- **Responsibilities:** Authenticates human actors and authorizes via Owner/Biller roles.
- **Ownership:** User.

9. Document Type Registry (Regulatory Runtime)

- **Responsibilities:** Holds canonical rules, required fields, and schema bounds per DTE Type.
- **Ownership:** DocumentTypeSpecification.
- **Forbidden:** Owning document instances or sequence states.

10. Configuration Management

- **Responsibilities:** Manages the single-issuer definition, establishments, and POS nodes.
- **Ownership:** IssuerConfiguration, Establishment, PointOfSale.

SECTION 4 — Dependency Rules

To prevent spaghetti architecture and circular logic, inter-module communication is governed by explicit rules:

- **Allowed Dependency Directions:**
 - **Issuance $\rightarrow$ Type Registry:** Issuance must read schema versions and validation rules from the Registry during document validation.
 - **Issuance $\rightarrow$ Catalog / Parties:** Issuance reads master data strictly to copy it into immutable snapshots during validation.
 - **Issuance $\rightarrow$ Evidence & Archive:** Issuance events (signing, acceptance) trigger Evidence to append immutable artifacts.
 - **Invalidation $\rightarrow$ Issuance:** Upon an invalidation event being accepted, Invalidation commands Issuance to transition the target document to Invalidated.
 - **All State-Changing Modules $\rightarrow$ Audit Trail:** Every module executing a state-mutating command must push a record to the Audit Trail.
- **Forbidden Dependencies:**
 - Master data modules (Catalog, Parties) must *never* reference or depend on Issuance modules or historical document contexts.
 - Evidence & Archive must *never* read from or depend on master data.

SECTION 5 — Command Processing Architecture

Commands represent explicit intents to mutate system state (e.g., ValidateDocument, SignDocument).

- **Command Lifecycle:** Every command passes through a strict gateway: Authentication $\rightarrow$ RBAC Authorization $\rightarrow$ Business Precondition Evaluation $\rightarrow$ Transactional State Mutation $\rightarrow$ Tamper-Evident Audit Generation $\rightarrow$ Persistence.
- **Validation Flow:** Validation logic is evaluated synchronously. A document's totals must perfectly reconcile with line items and taxes before it can exit the Draft state.
- **Transaction Flow:** Cross-aggregate references are handled strictly by identity (UUID). Database constraints (e.g., locking a document post-validation) enforce state consistency physically.
- **Audit Generation Flow:** Commands must orchestrate the creation of a tamper-evident audit entry *before* confirming success to the caller. If the audit entry fails to persist, the entire command transaction rolls back.
- **Evidence Generation Flow:** Evidence is generated as a side effect of irreversible domain

facts. Once SignDocument is invoked, the payload is frozen, and the JSON payload and PDF representation are securely written to the Evidence module.

SECTION 6 — Query Architecture

Queries bypass domain logic entirely to provide highly performant, read-only data access.

- **Query Flow:** Because the system employs CQRS, queries directly access the PostgreSQL database, reading from denormalized JSONB snapshots (e.g., issuer_snapshot, receptor_snapshot) directly embedded within transactional tables. This completely eliminates complex table joins when retrieving historical records.
- **Reporting Flow:** Monthly tax filings (Libro de Ventas) and internal dashboards query the fiscal_document table directly, leveraging indexing on issue_date and status.
- **Evidence Retrieval Flow:** Requests for compliance proofs (PDFs, MH API responses) query the evidence_artifact table directly. The system retrieves the immutable BYTEA blobs and re-verifies the cryptographic hash upon extraction to prove zero tampering occurred at rest.

SECTION 7 — Integration Architecture

V1 is built for human UI interaction, but the API boundaries are constructed to cleanly ingest traffic from an Enterprise Resource Planning (ERP) system.

- **ERP Document Ingestion:** ERPs invoke the CreateDraftDocument and ValidateDocument endpoints. Integrators must supply idempotency keys to ensure network retries do not result in duplicate Draft instances or exhausted control numbers.
- **Master Data Synchronization:** Counterparty and Item records can be updated continuously via batch integrations. The architecture guarantees that synchronizing this data will never retroactively modify historical invoices due to the strict enforcement of the Snapshot Law.
- **Directional Separation:** Outbound issuance requests are strictly segregated from inbound document reception (deferred to future scopes). Conflation of fiscal directions is structurally prohibited.

SECTION 8 — Regulatory Architecture

Regulatory rules are dynamic and must never be bound to application code deployment lifecycles.

- **Regulatory Bundles:** New tax behaviors, schema versions, and validation constraints are distributed by the vendor as cryptographically signed RegulatoryBundles.
- **Catalog Evolution:** Ministry of Finance catalogs evolve strictly through effective dating (valid_from, valid_to). The system resolves valid catalog options by performing a point-in-time lookup using the document's issuance date. Deprecated codes remain untouched in the database to satisfy the integrity of historical proofs.

SECTION 9 — Signing Architecture

The signing architecture guarantees Customer Sovereignty; the platform mathematically

proves intent without ever holding the raw cryptographic materials.

- **Trust Boundary:** The application never loads the .pfx or private key directly into application memory.
- **Signing Workflow:** When the SignDocument command fires, the Issuance module generates the canonical JSON payload. It utilizes a `certificate_reference` (URI/Vault pointer) stored in the configuration to hand the payload off to the customer's secure local vault or HSM.
- **Payload Freeze:** The vault performs the cryptographic signature and returns the signed payload. The application permanently freezes this data, immediately migrating it to the Evidence Bundle.

SECTION 10 — Transmission Architecture

Interacting with the MH API relies strictly on asynchronous, queue-based mechanics to prevent blocking human workflows during network degradation.

- **Communication Flow:** The TransmitDocument command places the signed document into a SUBMITTED state and queues the payload via a distributed queue (e.g., BullMQ). Queue workers retrieve the payload and manage the HTTP request to the MH.
- **Idempotency & Concurrency:** Distributed locks (e.g., Redis locks) prevent multiple queue workers from concurrently transmitting the same document.
- **Contingency Interaction:** If a Contingency Episode is open, the routing architecture overrides standard transmission behavior. Documents are flagged as CONTINGENCY_HELD rather than SUBMITTED and bypass the queue until the episode enters reconciliation.
- **Transmission Recording:** Every byte sent to and received from the MH—whether success, failure, or timeout—is recorded as an append-only entry in the TransmissionRecord. Outcomes are interpreted explicitly from the MH response, never optimistically inferred.

SECTION 11 — Audit & Evidence Architecture

The system guarantees legal defensibility through cryptographic proofs independent of the operational database state.

- **Evidence Creation:** Evidence artifacts (JSON payloads, PDFs, MH stamps) are inserted as BYTEA blobs. The application calculates a SHA-256 hash at the exact moment of capture. Once written, they are locked; the database role natively lacks deletion capabilities.
- **Audit Creation:** The AuditTrail maintains a chronological ledger of events. The `current_hash` of each new entry includes the `previous_hash`, the timestamp, the action, and the `actor_identity`.
- **Traceability:** A seamless link exists for auditors: Audit Entry $\rightarrow$ Target Fiscal Document $\rightarrow$ Evidence Bundle. This proves *who* initiated the action, *what* the fiscal impact was, and provides the exact cryptographic artifact submitted to the authority.

SECTION 12 — Failure Handling Architecture

Failure paths are definitively tracked rather than silently retried or deleted.

- **Validation Failures:** Mismatched totals or incomplete tax formats yield immediate synchronous HTTP 400/422 responses. The document remains safely in Draft.
- **Signing Failures:** If the HSM or Key Vault is unreachable, the system halts. The document remains in Validated status.
- **Transmission Failures (Transient):** Network timeouts result in queue-level retries. The document remains SUBMITTED, and the exact network error is appended to the TransmissionRecord.
- **Transmission Failures (Terminal):** If the MH issues a formal rejection (RECHAZADO), the document enters the terminal Rejected state. The assigned Control Number is considered permanently consumed and is never re-issued. Correction requires generating an entirely new document.

SECTION 13 — Evolution Strategy

V1 implements a closed scope , but its architectural seams are purposefully designed to support future enterprise expansion without refactoring core components.

- **Future DTE Types:** V1 hardcodes only 01 and 03 support , but all issuance rules are read from the DocumentTypeRegistry. Introducing new DTE types merely requires inserting new regulatory bounds into the configuration data, requiring zero code changes in the core engine.
- **Workflow Approvals:** The strict gateway between Draft and Validated is designed for interception. Future releases can easily route high-value documents to an ApprovalRequest entity before permitting the state transition.
- **Service Accounts:** The app_user identity structure utilizes unique UUIDs. V1.5+ programmatic integrations will inject API keys as long-lived JWTs that map into this exact structure, secured by a future policy engine.
- **Inbound DTE Reception:** Outbound contexts (Issuance) are structurally isolated. Future inbound processing logic will inhabit a net-new context (ReceivedDocument), guaranteeing zero risk of conflating accounts payable with accounts receivable logic.